

《人工智能基础 A》实验报告

CNN 图像识别与路标分类

学号

姓名：董卓然

一、实验报告（实验文档中只包含此部分即可，将截图放在此处）

1. 梯度下降【10】

1.1. 损失函数定义【2】

[1]:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import animation

def loss_function(x, y):
    return (x - 2) ** 2 + (y - 3) ** 2

# 测试
print("L(0,0) =", loss_function(0, 0)) # 期望: 13
print("L(2,3) =", loss_function(2, 3)) # 期望: 0 (最小值)
```

L(0,0) = 13

L(2,3) = 0

1.2. 梯度计算【3】

[2]:

```
def gradient(x, y):
    partial_x = 2 * (x - 2)
    partial_y = 2 * (y - 3)
    return partial_x, partial_y

# 测试
print("gradient(2,3) =", gradient(2, 3)) # 期望: (0, 0)
print("gradient(0,0) =", gradient(0, 0)) # 期望: (-4, -6)
```

gradient(2,3) = (0, 0)

gradient(0,0) = (-4, -6)

1.3. 梯度下降算法【3】

[3]:

```
def gradient_descent(x, y, learning_rate=0.1, num_iterations=50):
    trajectory = [(x, y, loss_function(x, y))]
    for _ in range(num_iterations):
        grad_x, grad_y = gradient(x, y)
        x -= learning_rate * grad_x
        y -= learning_rate * grad_y
        trajectory.append((x, y, loss_function(x, y)))
    return np.array(trajectory)

# 初始化参数
x, y = 0, 0
lr = 0.1
num_iters = 50
trajectory = gradient_descent(x, y, learning_rate=lr, num_iterations=num_iters)

print(f"起始点: ({trajectory[0,0]:.4f}, {trajectory[0,1]:.4f}), Loss={trajectory[0,2]:.4f}")
print(f"终止点: ({trajectory[-1,0]:.4f}, {trajectory[-1,1]:.4f}), Loss={trajectory[-1,2]:.4f}")
print(f"目标点: (2, 3), Loss=0")
```

起始点: (0.0000, 0.0000), Loss=13.0000
 终止点: (2.0000, 3.0000), Loss=0.000000
 目标点: (2, 3), Loss=0

1.4. 可视化【2】

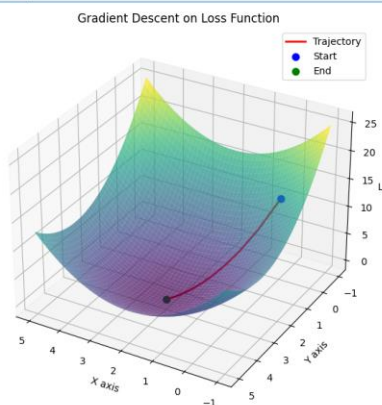
```
[4]: # 生成1000组数据
x_vals = np.linspace(-1, 5, 400)
y_vals = np.linspace(-1, 5, 400)
X, Y = np.meshgrid(x_vals, y_vals)
Z = loss_function(X, Y)

# 创建图形
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

# 绘制固定曲面
ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.7)
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_zlabel('Loss')
ax.set_title('Gradient Descent on Loss Function')
ax.view_init(elev=30, azim=120)

# 绘制梯度下降轨迹 (静态版, 需调用)
ax.plot(trajectory[:,0], trajectory[:,1], trajectory[:,2],
        color='red', linestyle='solid', label='Trajectory')
ax.scatter([trajectory[0,0]], [trajectory[0,1]], [trajectory[0,2]],
           color='blue', s=50, label='Start')
ax.scatter([trajectory[-1,0]], [trajectory[-1,1]], [trajectory[-1,2]],
           color='green', s=50, label='End')

ax.legend()
plt.tight_layout()
plt.show()
```



2. 基于深度学习的机器视觉-路标识别【80】

2.1. 实验数据集

2.1.1. 介绍数据集

2.1.2. 划分验证集【5】

```
[5]: import glob, os, random, cv2
import matplotlib.pyplot as plt

data_path = r"C:\Users\lenovo\Desktop\cnn_lab_data"
img_list = glob.glob(os.path.join(data_path, '*/*/*.png'))
print("数据集总数量:", len(img_list))

for i, img_path in enumerate(random.sample(img_list, 6)):
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    plt.subplot(2, 3, i + 1)
    plt.imshow(img)
    plt.axis('off')
plt.suptitle('Random Samples from GTSRB Dataset')
plt.tight_layout()
plt.show()
```

```
plt.show()
数据集总数量: 39209
```

Random Samples from GTSRB Dataset



2.2. 数据预处理与数据增强【5】

```
[5]: from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt

# 原始变换
data_transforms = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.3337, 0.3864, 0.3171),
                        (0.2672, 0.2564, 0.2629))
])

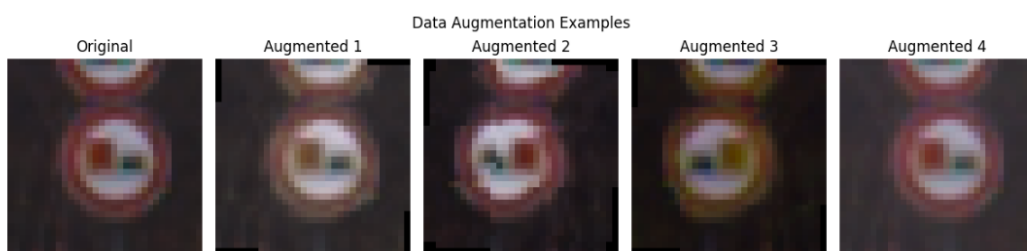
# 数据增强变换
data_jitter = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ColorJitter(brightness=0.4, contrast=0.4,
                          saturation=0.4, hue=0.1),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.3337, 0.3864, 0.3171),
                        (0.2672, 0.2564, 0.2629))
])

# 随机选择图片并显示
import glob, random
img_list = glob.glob(r"C:\Users\lenovo\Desktop\cnn_lab_data\train_images\*.png")
img_path = random.choice(img_list)
img = Image.open(img_path).convert('RGB')

# 显示原图 vs 4种增强效果
plt.figure(figsize=(12, 3))
plt.subplot(1, 5, 1)
plt.imshow(img.resize((32, 32)))
plt.title('Original')
plt.axis('off')

for i in range(4):
    aug = data_jitter(img)
    aug_img = aug.permute(1, 2, 0).numpy()
    # 统计均值
    mean = [0.3337, 0.3864, 0.3171]
    std = [0.2672, 0.2564, 0.2629]
    aug_img = aug_img * std + mean
    aug_img = aug_img.clip(0, 1)
    plt.subplot(1, 5, i+2)
    plt.imshow(aug_img)
    plt.title(f'Augmented {i+1}')
    plt.axis('off')

plt.suptitle('Data Augmentation Examples')
plt.tight_layout()
plt.show()
```



[]:

2.3. 定义网络架构【5】

```
(ai_lab3) C:\Users\lenovo\Desktop\cnn_lab_data>python train.py
[INFO] 学号: 3250102672
[INFO] 数据路径: C:\Users\lenovo\Desktop\cnn_lab_data
[INFO] GPU可用: True
[INFO] 加载数据集...
[INFO] 训练集大小: 70678
[INFO] 验证集大小: 3870
[INFO] 初始化模型 (ResNet18)...
Net(
  (model): ResNet(
    (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
    (layer1): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
      (1): BasicBlock(
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (layer2): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (downsample): Sequential(
          (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (layer3): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (downsample): Sequential(
          (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
)
```

```

s=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stat
s=True)
    )
  )
  (1): BasicBlock(
    (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stat
s=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stat
s=True)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=43, bias=True)
)
)
Epoch 1 [Train]: 100%|████████████████████| 553/553 [02:00<00:00, 4.59it/s]

Training set: Avg loss: 0.4432, Accuracy: 61048/70678 (86.4%)

[Validation]: 100%|████████████████████| 31/31 [00:04<00:00, 6.83it/s]

Validation set: Avg loss: 0.9324, Accuracy: 3048/3870 (78.8%)

[INFO] 已保存模型: C:\Users\lenovo\Desktop\cnn_lab_data\results\model_1.pth
[INFO] 最优模型已更新 (val_acc=78.8%)
Epoch 2 [Train]: 100%|████████████████████| 553/553 [02:03<00:00, 4.47it/s]

Training set: Avg loss: 0.1173, Accuracy: 68071/70678 (96.3%)

[Validation]: 100%|████████████████████| 31/31 [00:04<00:00, 6.86it/s]

Validation set: Avg loss: 0.5777, Accuracy: 3271/3870 (84.5%)

```

2.4. 训练网络

2.4.1. 训练函数和验证函数【5】

```

# -----
# 训练函数
# -----
def train(epoch, model, optimizer, train_loader, use_gpu):
    model.train()
    correct = 0
    training_loss = 0

    for data, target in tqdm(train_loader,
                              desc=f"Epoch {epoch} [Train]"):
        data, target = Variable(data), Variable(target)
        if use_gpu:
            data, target = data.cuda(), target.cuda()

        optimizer.zero_grad()
        output = model(data)
        loss = F.cross_entropy(output, target, reduction="sum")
        loss.backward()
        optimizer.step()

        pred = output.max(dim=1)[1]
        correct += (pred == target).sum().item()
        training_loss += loss.item()

    avg_loss = training_loss / len(train_loader.dataset)
    accuracy = 100. * correct / len(train_loader.dataset)
    print(f'\nTraining set: Avg loss: {avg_loss:.4f}, '
          f'Accuracy: {correct}/{len(train_loader.dataset)} ({accuracy:.1f}%)')
    return avg_loss, accuracy
# -----

```

```

# 验证函数
#
def validation(model, val_loader, use_gpu):
    model.eval()
    val_loss = 0
    correct = 0

    with torch.no_grad():
        for data, target in tqdm(val_loader, desc="[Validation]"):
            data, target = Variable(data), Variable(target)
            if use_gpu:
                data, target = data.cuda(), target.cuda()

            output = model(data)
            val_loss += F.cross_entropy(output, target,
                                       reduction="sum").item()
            pred = output.max(dim=1)[1]
            correct += (pred == target).sum().item()

    avg_loss = val_loss / len(val_loader.dataset)
    accuracy = 100. * correct / len(val_loader.dataset)
    print(f'\nValidation set: Avg loss: {avg_loss:.4f}, '
          f'Accuracy: {correct}/{len(val_loader.dataset)} ({accuracy:.1f}%)\n')
    model.train()
    return avg_loss, accuracy

```

2.4.2. 训练超参设置【5】

```

from tqdm import tqdm
import matplotlib.pyplot as plt

#
# 超参数设置
#
class args:
    data      = r"C:\Users\lenovo\Desktop\cnn_lab_data"
    batch_size = 128
    epochs    = 20
    lr        = 0.0005
    seed      = 1
    save_dir  = r"C:\Users\lenovo\Desktop\cnn_lab_data\results"

torch.manual_seed(args.seed)

```

2.4.3. 开始训练【20】

```

for epoch in range(1, args.epochs + 1):
    tr_loss, tr_acc = train(epoch, model, optimizer,
                             train_loader, use_gpu)
    va_loss, va_acc = validation(model, val_loader, use_gpu)

    res["loss"].append(tr_loss)
    res["accuracy"].append(tr_acc)
    res["val_loss"].append(va_loss)
    res["val_accuracy"].append(va_acc)

    scheduler.step(va_loss)

    # 每个epoch保存
    ckpt = os.path.join(args.save_dir, f'model_{epoch}.pth')
    torch.save(model.state_dict(), ckpt)
    print(f'[INFO] 已保存模型: {ckpt}')

    # 保存最优模型
    if va_acc > best_val_acc:
        best_val_acc = va_acc
        best_path = os.path.join(args.save_dir, 'best_model.pth')
        torch.save(model.state_dict(), best_path)
        print(f'[INFO] 最优模型已更新 (val_acc={va_acc:.1f}%)')

print(f'\n[INFO] 训练完成, 总耗时: {time.time()-start:.1f}s')
print(f'[INFO] 最高验证集准确率: {best_val_acc:.1f}%)

```

```

ue))
KeyboardInterrupt
^C
(ai_lab3) C:\Users\lenovo\Desktop\cnn_lab_data>notepad train.py

(ai_lab3) C:\Users\lenovo\Desktop\cnn_lab_data>python train.py
[INFO] 学号: 3250102672
[INFO] 数据路径: C:\Users\lenovo\Desktop\cnn_lab_data
[INFO] GPU可用: True
[INFO] 加载数据集...
[INFO] 训练集大小: 70678
[INFO] 验证集大小: 3870
[INFO] 初始化模型 (ResNet18)...
Net(
  (model): ResNet(
    (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
    (layer1): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
      (1): BasicBlock(

```

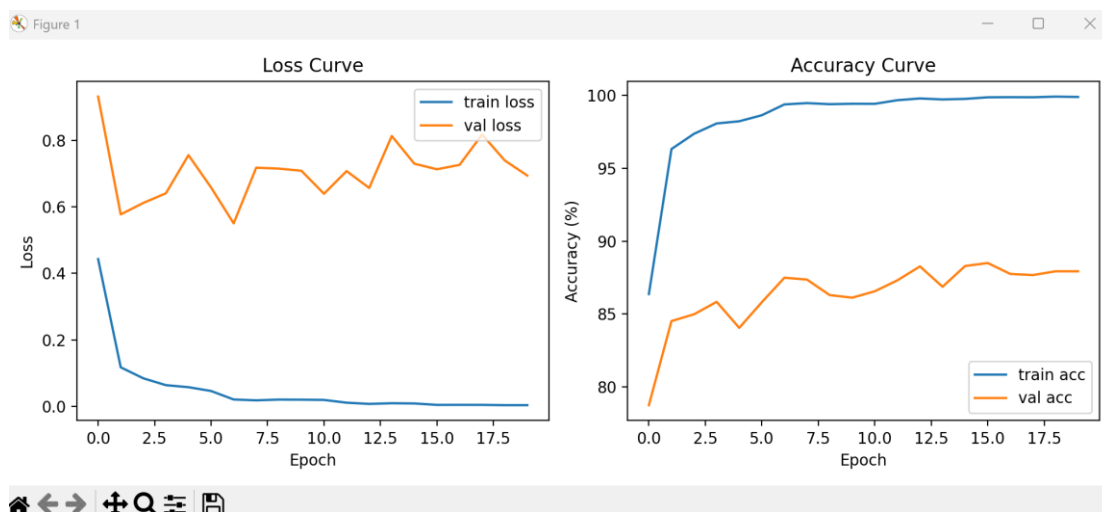
```
)
)
Epoch 1 [Train]: 100%|██████████| 553/553 [02:00<00:00, 4.59it/s]
Training set: Avg loss: 0.4432, Accuracy: 61048/70678 (86.4%)
[Validation]: 100%|██████████| 31/31 [00:04<00:00, 6.83it/s]
Validation set: Avg loss: 0.9324, Accuracy: 3048/3870 (78.8%)
[INFO] 已保存模型: C:\Users\lenovo\Desktop\cnn_lab_data\results\model_1.pth
[INFO] 最优模型已更新 (val_acc=78.8%)
Epoch 2 [Train]: 100%|██████████| 553/553 [02:03<00:00, 4.47it/s]
Training set: Avg loss: 0.1173, Accuracy: 68071/70678 (96.3%)
[Validation]: 100%|██████████| 31/31 [00:04<00:00, 6.86it/s]
Validation set: Avg loss: 0.5777, Accuracy: 3271/3870 (84.5%)
[INFO] 已保存模型: C:\Users\lenovo\Desktop\cnn_lab_data\results\model_2.pth
[INFO] 最优模型已更新 (val_acc=84.5%)
Epoch 3 [Train]: 100%|██████████| 553/553 [02:03<00:00, 4.48it/s]
Training set: Avg loss: 0.0842, Accuracy: 68811/70678 (97.4%)
[Validation]: 100%|██████████| 31/31 [00:04<00:00, 6.89it/s]
Validation set: Avg loss: 0.6122, Accuracy: 3289/3870 (85.0%)
[INFO] 已保存模型: C:\Users\lenovo\Desktop\cnn_lab_data\results\model_3.pth
[INFO] 最优模型已更新 (val_acc=85.0%)
Epoch 19 [Train]: 100%|██████████| 553/553 [01:37<00:00, 5.69it/s]
Training set: Avg loss: 0.0034, Accuracy: 70611/70678 (99.9%)
[Validation]: 100%|██████████| 31/31 [00:03<00:00, 8.73it/s]
Validation set: Avg loss: 0.7402, Accuracy: 3403/3870 (87.9%)
[INFO] 已保存模型: C:\Users\lenovo\Desktop\cnn_lab_data\results\model_19.pth
Epoch 20 [Train]: 100%|██████████| 553/553 [01:37<00:00, 5.66it/s]
Training set: Avg loss: 0.0034, Accuracy: 70594/70678 (99.9%)
[Validation]: 100%|██████████| 31/31 [00:03<00:00, 8.25it/s]
Validation set: Avg loss: 0.6947, Accuracy: 3403/3870 (87.9%)
[INFO] 已保存模型: C:\Users\lenovo\Desktop\cnn_lab_data\results\model_20.pth
[INFO] 训练完成, 总耗时: 3272.2s
[INFO] 最高验证集准确率: 88.5%
[INFO] 训练曲线已保存至 C:\Users\lenovo\Desktop\cnn_lab_data\results\training_curve.p
```

2.4.4. 模型训练过程图形化【20】

```
def plot_training_history(res, save_dir):
    # 损失曲线
    plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.plot(res['loss'], label='train loss')
    plt.plot(res['val_loss'], label='val loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Loss Curve')
    plt.legend(loc='upper right')

    # 准确率曲线
    plt.subplot(1, 2, 2)
    plt.plot(res['accuracy'], label='train acc')
    plt.plot(res['val_accuracy'], label='val acc')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy (%)')
    plt.title('Accuracy Curve')
    plt.legend(loc='lower right')

    plt.tight_layout()
    fig_path = os.path.join(save_dir, 'training_curve.png')
    plt.savefig(fig_path)
    print(f'[INFO] 训练曲线已保存至 {fig_path}')
    plt.show()
#
```



2.5. 加载模型和模型评估【15】

```
# -----
# 加载最佳模型
# -----
model_path = r"C:\Users\lenovo\Desktop\cnn_lab_data\results\best_model.pth"

model = Net(nclasses=43)
model_state = torch.load(model_path, map_location=device)
model.load_state_dict(model_state)
model = model.to(device)
model.eval()
print(f"[INFO] 模型加载成功: {model_path}")

# -----
# 图像预处理
# -----
```

```

7 # 预测函数（接口固定，不要修改参数和返回值）
8 #
9 def predict(img):
10     """
11     加载模型和模型预测
12     :param img: PIL.Image 对象 或 图片路径字符串
13     :return: int, 模型识别图片的类别
14     """
15     if isinstance(img, str):
16         img = Image.open(img).convert('RGB')
17
18     img_tensor = data_transforms(img)      # [3, 32, 32]
19     img_tensor = img_tensor.unsqueeze(0)    # [1, 3, 32, 32]
20     img_tensor = img_tensor.to(device)
21
22     with torch.no_grad():
23         output = model(img_tensor)
24         y_predict = output.data.max(1, keepdim=True)[1].item()
25
26     return y_predict

```

```

(base) C:\Users\lenovo\Desktop\cnn_lab_data>conda activate ai_lab3
(ai_lab3) C:\Users\lenovo\Desktop\cnn_lab_data>python predict.py
[INFO] 模型加载成功: C:\Users\lenovo\Desktop\cnn_lab_data\results\best_model.pth
[INFO] 学号: 3250102672
[INFO] 图片路径: C:\Users\lenovo\Desktop\cnn_lab_data\val_images\0\00000_00000_00000.png
[INFO] 预测类别: 1
(ai_lab3) C:\Users\lenovo\Desktop\cnn_lab_data>

```